

IIT dhArwAD
Midsem Exam:(25/09/2024)
CS101: Computer Programming
Course instructors: Dr. Siba Swain and Dr. Jivnesh Sandhan

Name:

Roll No:

Department:

Instructions:

1. Total duration: 2 hours. Please write your name, roll number and department.
2. This booklet has pages (9 pages + 1 pages for rough work). No part of your answers should be on pages designated for rough work.
3. Write/mark your answers clearly in the provided space. Please keep your answers precise and concise.
4. Answer each question using the information provided in the question (**no clarifications during the exam**). If you want to make any assumptions, please state them in your answer.

Question:	1	2	3	4	Total
Points:	6	10	18	16	50
Score:					

1. True (T) or False (F). Justify in a few words. You can also write a corrected statement as a justification. No marks without correct justification.

(a) (1 point) ____ It is possible to have less *if* than *else* occurrences in correctly compiling program.

1a - 1f:
0 or 1, no partial marks
Justify: **False, if-else block happens starting with the if block and then else block, so there should be either the same number of if-else blocks, or more if blocks than else blocks.**

(b) (1 point) ____ The following code does not print anything.

```
1 # include <stdio.h>
2 int main ()
3 {
4     int c = 0, d = 1;
5     if (!(++c && d--)) printf( ' 'Welcome' ');
6     return 0;
7 }
```

Listing 1: True or false.

Justify: **True, because the if condition (++c && d--) with initial values as c = 0, and d = 1 will be (1 && 1) = 1 and !(1) = 0, if condition fails, thus no output is printed for the program**

(c) (1 point) ____ A *static* variable declared in a custom function is also visible to other functions.

Justify: **False. It is private to the function where it belongs. Other, functions cannot see it.**

(d) (1 point) ____ A global variable can be accessed by all functions in the program.

Justify: **True, because the scope of the global variable is throughout the entire program**

(e) (1 point) ____ *break* helps to skip the rest of the body of loop and *continue* helps to exit loop.

Justify: **False, break statement brings the control out of the inner for loop, where as continue proceeds with the next iteration of the loop**

(f) (1 point) ____ The float expressions can't be tested in switch.

Justify: **True, because the IEEE floating point standards have incorrect mismatch. Meaning 0.3 cannot be compared with 0.3 itself as it is compared with a value close to 0.3 and not actually 0.3**

2. More than one correct question. Partial marking is there provided you don't select incorrect option:

0.5 each (a) (2 points) Identify the invalid variable names.

☐ A_3 ☐ 5_A ☐ abc@123 ☐ no-entry ☐ return

Justify: **Invalid variable names are: abc@123 (use of special character other than underscore), no-entry (use of special character other than underscore), 5_A (a variable name cannot start with a number), and return (keyword)**

(b) (2 points) Select all the correct options regarding arrays.

- ☐ All the characters of a string are stored in successive memory locations.
- ☐ If an array is declared as `int arr[50]`, it holds 49 elements;
- ☐ A two-dimensional array contains data of two different types.
- ☐ An array stores all its data elements in non-consecutive memory locations.

Justify: **Option 1 is correct as string is an array of characters with all characters stored in successive memory locations, Option 2 is incorrect because, `int arr[50]` contains 50 blocks stored for integer type values, however the indices range from 0 to 49, Option 3 is incorrect because either it is 1-D or 2-D array, all the elements need to be of same data type, Option 4 is incorrect because array contains all its elements in consecutive memory locations**

(c) (2 points) Select all the correct options for this given code

1. if option 1 and 3 selected, then 2 marks

2. if either 1 or 3 selected then 1 mark

3. if 1, 3 and any other selected then 0.5 marks

4. if 1 and 3 not selected then 0

```
1  # include <stdio.h>
2  int main () {
3      int a, b;
4      c = a + b;
5      a = 2;
6      b = 1;
7      printf( "Result is %d" , c );
8      return 0; }
```

Listing 2: Verify if the code is valid.

- ☐ This code will not compile.
- ☐ If we declare the 'c' variable, it will compile and print its value as 3.
- ☐ If we declare the 'c' variable, it will compile and print its value as garbage.
- ☐ None of the above.

Justify: **c variable is not declared, so compilation error, but even it is declared, it will print a garbage value. So, options 1 and 3 are correct**

(d) (2 points) Analyze the following code and select all the correct options for this given code

1. if options 1 and 2 selected then 2 marks

2. if either 1 or 2 selected then 1 mark

3. if 1, 2 and any other selected then 0.5 marks

4. if 1 and 2 not selected then 0 marks

```
1 #include <stdio.h>
2 void swap(int a, int b)
3 {
4     int temp = a;
5     a = b;
6     b = a;
7 }
8 int main()
9 {
10    int a = 10, b = 5;
11    printf("Before swapping, a = %d, and b = %d\n", a, b);
12    swap(a, b);
13    printf("After swapping, a = %d, and b = %d\n", a, b);
14    return 0;
15 }
```

Listing 3: Analyze the code.

- ☐ Before swapping, a = 10, and b = 5
- ☐ After swapping, a = 10, and b = 5
- ☐ After swapping, a = 5, and b = 10.
- ☐ Before swapping, a = 5, and b = 10

Justify: Options 1 and 2 are correct, as the arguments are passed as values and not references, so a and b variable are not swapped

(e) (2 points) Which of the following case declarations would be illegal in a switch case statement?

1. if 3 selected or 1, and 3, selected then 2 marks

2. if either 1 or 3 selected then 1 mark

3. if all selected then 0.5 marks

4. if none selected then 0 marks

- ☐ case 6: printf("The case is 6")
- ☐ case 'a': printf("The case is a"); break;
- ☐ case (float)7: printf("The case is 7.0"); break;
- ☐ case (char)10: printf("The case is %c", 10);

Justify: [1] Incorrect. Syntax error: missing " and ;
OR students consider as printing mistake - award full marks for [1]
[2] valid: Chars are allowed due to ASCII one to one integer correspondence.
[3] Incorrect: Floats are banned in switch
[4] Valid: Chars are allowed due to ASCII one to one integer correspondence

3. Short/medium-length answer questions:

(a) (2 points) Complete the code snippet by filling the the correct logic on the dotted line to get the following output.

2 marks if any of the solutions provided is written, else 0 marks

Output: 1 3 5

```

1 #include <stdio.h>
2 int main()
3 {
4     for (int i = 1; i <= 5; i++)
5     {
6         if(i%2 != 0) ----- // Write your logic here
7
8         printf("%d ", i);
9     }
10    return 0;
11 }

```

or, the following code also works

if(i%2==0)
continue;

or,
if(i%2) continue;

or,
if(i%2 == 1)

Listing 4: Fill in the blanks.

(b) (4 points) Write the output of the following program in an appropriate box.

based on
solution
provided,
if any of these
meet then full
marks,

else, 1 mark
each

```

1 #include <stdio.h>
2 int main()
3 {
4     float a = 5, b = 2.0;
5     printf("%.2f\n", a);
6     printf("%d %.2f\n", a, b);
7     printf("%d\n", (int) a/b);
8     printf("%.1f\n", (int) a/3);
9     return 0;
10 }

```

Line 5: 5.00

Line 6: warning/error
message for a, 2.00
OR
garbage_value 0.00
OR
garbage_value 5.00

Line 7: warning/error
message
OR
garbage_value

Listing 5: Fill in the blanks.

Line No	Output
5	
6	
7	
8	

Line 8: warning/error
message
OR
2.5

(c) (3 points) Complete the code snippet by writing the correct logic to get the following output.

Output:

```

1
12
123
1234

```

3 marks if any of
the given
solutions is
written,

else deduct 1
mark for each
syntax error

```

1 #include <stdio.h>
2 int main()
3 {
4     int i = 1, j = 1;
5     for(i=1; i<5; i++)
6     {
7         for( j=1; j<5; j++) \\ Write your logic here.
8         {
9             if(i>=j) \\ Write your logic here
10                printf("%d", j);
11        }
12    }
13    printf('\n');
14    return 0;
15 }

```

or, the following is also correct

```

for(j=1; j<=i; j++)
    printf("%d", j);

```

Listing 6: Fill in the blanks.

(d) (3 points) Complete the function to identify whether a string is palindrome or not.

3 marks if all 3
blanks are filled
correctly

else, deduct 1
mark for each
incorrect filling

```

1 #include <stdio.h>
2 #include <string.h>
3
4 int isPalindrome(char str[]) {
5     int low = 0;
6     int high = strlen(str) - 1;
7
8     // Keep comparing till low index crosses high index
9
10    while ( low<high )
11    {
12
13        if (str[low] != str[high])
14            return 0; // not a palindrome.
15
16        low = low + 1; // Update low index
17        high = high - 1; // Update high index
18
19    }
20
21    return 1; // is a palindrome
22 }
23

```

Another way,
low<strlen(str)-1

or,
low++;
high--;

Listing 7: Palindrome string.

(e) (2 points) Write the output of the program and justify your answer.

1 mark:
correct output
1 mark:
correct
justification

```

1 #include <stdio.h>

```

```

2  int main()
3  {
4      int n = 1;
5      switch(n)
6      {
7          case 0: printf("zero");
8          case 1: printf("one");
9          case 2: printf("two"); break;
10         case 3: printf("three");
11     }
12     return 0;
13 }

```

Listing 8: Guess the output.

Justify: **Output: onetwo**

Explanation: There is no break after case 1 so case 2 also will be executed due to falling through concept.

- (f) (4 points) Find the lines which will result in COMPILATION errors. For such line(s), write the line no. and also a correct version in the respective columns. The corrected program should compile.

1 mark for each
corrected line,
else 0 mark for
each part

```

1  #include <stdio.h>
2  int main()
3  {
4      int 1a = 5;
5      int b = 1
6      3 = b;
7      printf("%d %d", 1a, b);
8      return 0;
9  }

```

Listing 9: Fix the bugs.

Line No	Corrected line
4	int a1 = 5;
5	int b = 1;
6	b = 3, because assignment of b to 3 which is 'const' cannot be done
7	printf("%d %d", a1, b);

4. Long answers.

- (a) (6 points) The following program is supposed to read an English sentence with less than or equal to 99 characters, and then convert all lowercase English alphabets in the sentence to uppercase and vice versa. We promise the sentence will be given in a single line i.e. the sentence won't contain the newline character. Fill in the blanks using the directions provided in the comments.

Example:

INPUT: The Month is SePtEmber

OUTPUT: tHE mONTH IS sEpTeMBER

REFER last page for
solution

```
1 #include <stdio.h>
2 int main()
3 {
4     char str[100];
5     // Read the sentence from input as a string
6
7     -----
8
9     int i;
10    for(i=0; i<100; i++)
11        // Check if character at index i is in UPPER case
12
13        if(-----)
14
15            // Convert it into lower case.
16
17            str[i] = ----- ;
18
19            // Check if the character is in lower case
20
21            else if(-----)
22
23                // Convert it into UPPER case.
24
25                str[i] = ----- ;
26
27                // Otherwise it is not an alphabet.
28                // Check if this is end of the string.
29
30                else if(-----)
31
32                    break;
33    printf("%s",str);
34    return 0;
35 }
```

Listing 10: Switch the case.

1 mark for each
correct statement
written

if students use
built-in string
methods, then
deduct 2 marks
after the correct
evaluation

- (b) (6 points) Complete the following code that is meant to print hollow square given its side length (length > 2), using * as a basic building block. Complete the horzLine and vertLine functions in the respective boxes. For example, an input of 5 will print.

Output:

```
* * * * *
*       *
*       *
*       *
* * * * *
```

```
1 #include <stdio.h>
2 void horzLine(int);
3 void vertLine(int);
4 int main()
5 {
6     int n;
7     scanf("%d", &n);
8     horzLine(n);
9     vertLine(n-2);
10    horzLine(n);
11    return 0;
12 }
```

Listing 11: Print square.

1 mark argument
1 mark for loop
1 mark printing stars

```
void horzLine(int n) {
    for (int i = 0; i < n; i++) {
        printf("* ");
    }
    printf("\n");
}
```

or,
the following is also correct

```
void horzLine(int N)
{
    for(int i = 0; i<N; i++)
        printf("* ");
    printf("\n");
}
```

1 mark argument
1 mark for loop
0.5 marks printing stars
0.5 marks printing space

```
// Please note that N-2 is passed to this function.
void vertLine(int N)
{
    for(int j = 0; j<N; j++){
        printf("* ");
        for(int i = 0; i<N; i++)
            printf(" ");
        printf("*\n");
    }
}
```


- (c) (4 points) Complete the function to add two square matrices. The below program adds two square matrices of size $N \times N$. Assume that N is a constant and your code should work for any valid N .

each line 1
mark

```
1 #include <stdio.h>
2 int N;
3
4 // This function adds A[][] and B[][],
5 // and stores the result in C[][]
6 void add(int A[][N], int B[][N], int C[][N])
7 {
8     int i, j;
9     -----
10    for(int i=0; i<N; i++)
11    -----
12        for(int j=0; j<N; j++) {
13        -----
14            C[i][j] = A[i][j] + B[i][j];
15        -----
16        }
17    }
18 }
19
20
```

Listing 12: Add 2 squares.

Q4.a) NOTE: Since the given code in the question is complete, so if in the question header file string.h is not given it means that built-in string methods cannot be used

4.a) solution

```
#include <stdio.h>

int main() {
    char str[100];
    // Read the sentence from input as a string
    gets(str); // scanf("%s",str) is not valid here since space is there in string

    // if getline with correct syntax is used then its fine.
    int i;
    for(i=0; i<100; i++)
        // Check if character at index i is in UPPER case
        if(str[i]>='A' && str[i]<='Z')
            // Convert it into lower case.
            str[i] = str[i] + 'a' - 'A' ;

        // Check if the character is in lower case
        else if(str[i]>='a' && str[i]<='z')
            // Convert it into UPPER case.
            str[i] = str[i] + 'A' - 'a' ;

        // Otherwise it is not an alphabet.
        // Check if this is end of the string.
        else if(str[i]=='\0')
            break;

    printf("%s",str);

    return 0;
}
```

IIT dhArwAD Endsem Exam CS101: Computer Programming Course instructors: Dr. Siba Swain and Dr. Jivnesh Sandhan

Name:

Roll No:

Department:

Instructions:

1. Total duration: 3 hours. Please write your name, roll number and department.
2. This booklet has pages (10 pages).
3. Write/mark your answers clearly in the provided space. Please keep your answers precise and concise.
4. Answer each question using the information provided in the question (**no clarifications during the exam**). If you want to make any assumptions, please state them in your answer.

Question:	1	2	3	4	Total
Points:	6	14	20	35	75
Score:					

1. True (T) or False (F). Justify in a few words. You can also write a corrected statement as a justification. No marks without correct justification. No negative marking.

(a) (1 point) ____ If we declare two arrays as `int c[4]; int *d = (int*)malloc(16)` then the expressions `sizeof(c)` and `sizeof(d)` will give us the same value.

Justify: **False. `sizeof(c) = 16` and `sizeof(d) = 8` because it is a pointer.**

(b) (1 point) ____ `char str[10] = "Hello";` Then the following if statement will print YES :
`if(str[5] = '\0') printf("YES"); else printf("NO");`

Justify: **False. This is an assignment operator, not a comparison. This assignment returns the assigned value, which is `'\0'` (numerically 0)**

(c) (1 point) ____ If `struct node{ int a, b, c; };` is a structure and `struct node *ptr;` is a pointer to a node variable. Then `sizeof(ptr)` will give us the value 8.

Justify: **True, It will be 8 because any pointer always takes 8 bytes to store an address.**

(d) (1 point) ____ The space needed to store an array of 100 integers is the same as that needed to store a singly linked list `struct node{ int a; struct node* next; };` with 100 nodes.

Justify: **False. The space required by the singly linked list is significantly larger than that of the array because of the extra space needed for pointers in each node**

(e) (1 point) ____ If a pointer to the current node is given in the singly linked list, it is possible to insert a new node before it.

Justify: **True. In general, it is not possible except if the current node is the list's head.**

(f) (1 point) ____ `int c[100]; int *ptr = (int*)realloc(c, 200 * sizeof(int));` This will fail to make the size of c from 100 to 200.

Justify: **True. This will give an error because c is not a dynamic array. It is not allowed.**

2. There may be multiple correct answers. Justifications are not required. You will receive 2 marks for selecting all the correct options. If you skip the question, you get 0 marks.

Negative marking: If you miss any correct option or choose an incorrect one, a penalty of -1 mark will be applied.

- (a) (2 points) `struct Node {int val; struct Node *next;};` Using this structure, a singly linked list is created with 8 nodes. If given a pointer `struct Node *h;` to the head node of this list, which of the following expressions will correctly generate the value stored at the second node in the list?
- ☐ `(h->next).val` ☐ `(h.next)->val` ☐ `(* (h->next)).val` ☐ `(h->next)->val`

Answer: C and D. This is based on the fact that `*(h).next` is the same as `h->next` where `h` is a pointer.

- (b) (2 points) You need to store a sequence of odd numbers (1, 3, 5, ...) with the total number of elements unknown in advance. Which of the following data structures would be the most appropriate for accomplishing this task efficiently?

☐ Static Array ☐ Linked list ☐ Dynamic Array ☐ None of the above

Answer: B and C. Based on demand we can expand the nodes/memory with these structures.

- (c) (2 points) Assume you have a sequence of odd numbers as described in the (b). Which data structures would efficiently insert even numbers at specific positions between existing odd numbers?

☐ Array ☐ singly linked list ☐ doubly linked list ☐ dynamic array

Answer: B and C. This is one of the advantages of these structures compared to arrays.

- (d) (2 points) Select all the incorrect options.

- ☐ The structure size is equal to or greater than the total size of all its members.
☐ The size of the union is the size of its largest member.
☐ In union, only one member can contain data at the same time.
☐ None of the above.

Answer: D. All are correct facts.

- (e) (2 points) What will the following program's output be? Select the correct answers.

```
1 #include <stdio.h>
2 typedef char* ptr;
3 #define PTR char*
4 int main(){
5     ptr a, b, c;
6     PTR x, y, z;
7     printf("sizeof a:%ld, b:%ld, c:%ld and " ,sizeof(a),
8           sizeof(b),sizeof(c));
9     printf("sizeof x:%ld, y:%ld, z:%ld" ,sizeof(x),sizeof(y),
10           sizeof(z));
11     return 0;}
```

Listing 1: Guess the output.

- ☐ sizeof a:8, b:8, c:8 and sizeof x:8, y:1, z:1
☐ sizeof a:8, b:8, c:8 and sizeof x:1, y:1, z:1
☐ sizeof a:8, b:8, c:8 and sizeof x:8, y:8, z:8
☐ sizeof a:1, b:1, c:1 and sizeof x:8, y:8, z:8

Answer: A `typedef` will change the data type of all variables but `define` will do it for only first variable; because it simply copy paste the syntax.

(f) (2 points) What will the following program's output be? Select the correct answers.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 void printArray(int *ptr){
4     printf("The elements of the array are: ");
5     for (int i = 0; i < 5; ++i) printf("%d, ", ptr[i]);}
6
7 void initialiseArray(int *ptr){
8     ptr = (int*)malloc(5 * sizeof(int));
9     for (int i = 0; i < 5; ++i) ptr[i] = i + 1;}
10
11 int main(){
12     int* ptr;
13     initialiseArray(ptr);
14     printArray(ptr);
15     return 0;}
16
```

Listing 2: Guess the output.

- ☐ Output will be: "The elements of the array are: 1, 2, 3, 4, 5,"
- ☐ `ptr` is passed by value, the changes made in `initialiseArray()` won't affect the pointer in `main()`
- ☐ The program will result in undefined behavior when accessing uninitialized memory.
- ☐ The best way to fix the issue in this code use `int** ptr` as a parameter in `initialiseArray()` and pass the address of the pointer from `main()`

Answer: B, C and D. These options itself are the explanation of the error.

(g) (2 points) What does the C program do upon executing it?

```
1 #include <stdio.h>
2 void main()
3 {   int x, y, m, n;
4     scanf("%d %d", &x, &y);
5     /* x > 0 and y > 0 */
6     m = x; n = y;
7     while (m != n)
8     {   if(m > n) {m = m - n;}
9         else {n = n - m;}
10    }
11    printf("%d", n);
12 }
```

Listing 3: Guess the output.

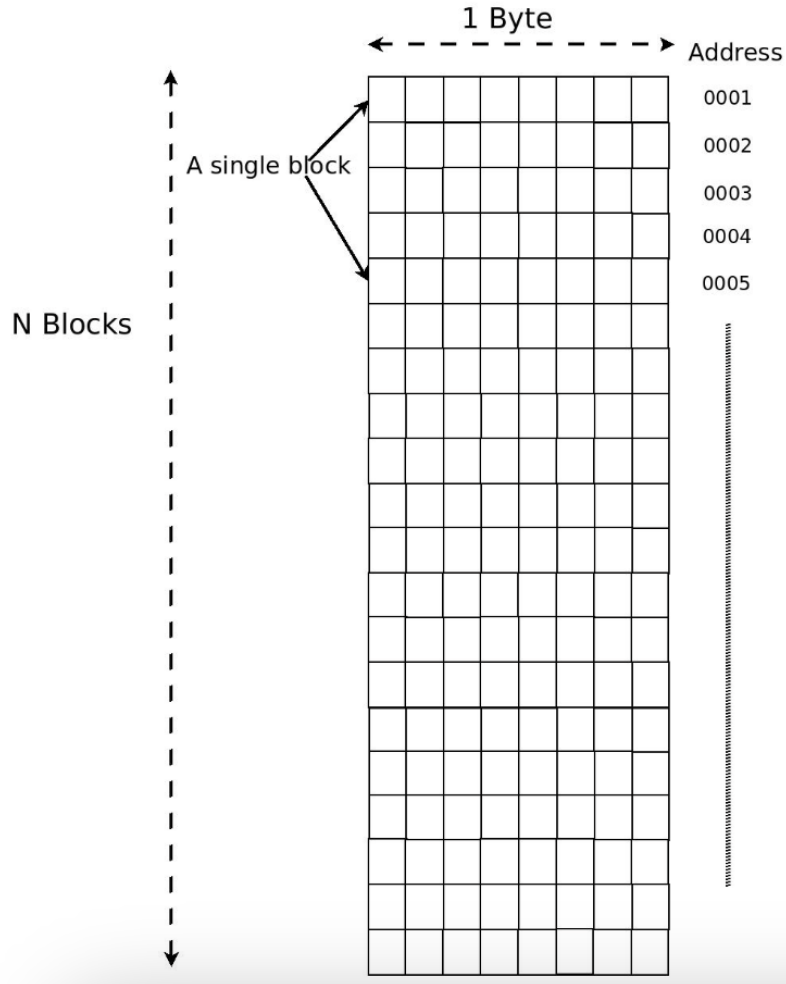
- ☐ To find the Least Common Multiple (LCM) of two numbers.

- ☐ To swap the values of two integers.
- ☐ To determine if two numbers are equal.
- ☐ None of the above.

Answer: D. To calculate the Greatest Common Divisor of two numbers.

3. Short/medium-length answer questions:

- (a) (6 points) Consider a snapshot of the computer memory below. It consists of N blocks of memory. Each memory block can hold 1 byte of information as shown in the figure. Note that pointer needs 8 bytes but for simplicity assume that it takes only 1 byte to store an address. Each row is a single byte illustrated by the arrows.



Consider the following code snippet:

```
#include <stdio.h>
int main(){
char **ptrArr = (char**) malloc(3* sizeof(char*)); //Line 1
for(int i = 0; i < 3; i++){ //Line 2
    ptrArr[i] = (char*) malloc((i+1)* sizeof(char)); //Line 3
} //Line 4
}
```

Listing 4: Code

(a) After executing Line 1, explain how the blocks of memory will be occupied for ptrArr. You need to clearly label and show the places in the memory block diagram above which will be affected by Line 1.

Line 1 will reserve memory spaces equal to the number of rows of the two d array - ptrArr[0], ptrArr[1], and ptrArr[2]. These memory spaces will be contiguous. Since ptrArr is a double pointer, therefore it will store the address of ptr[0].

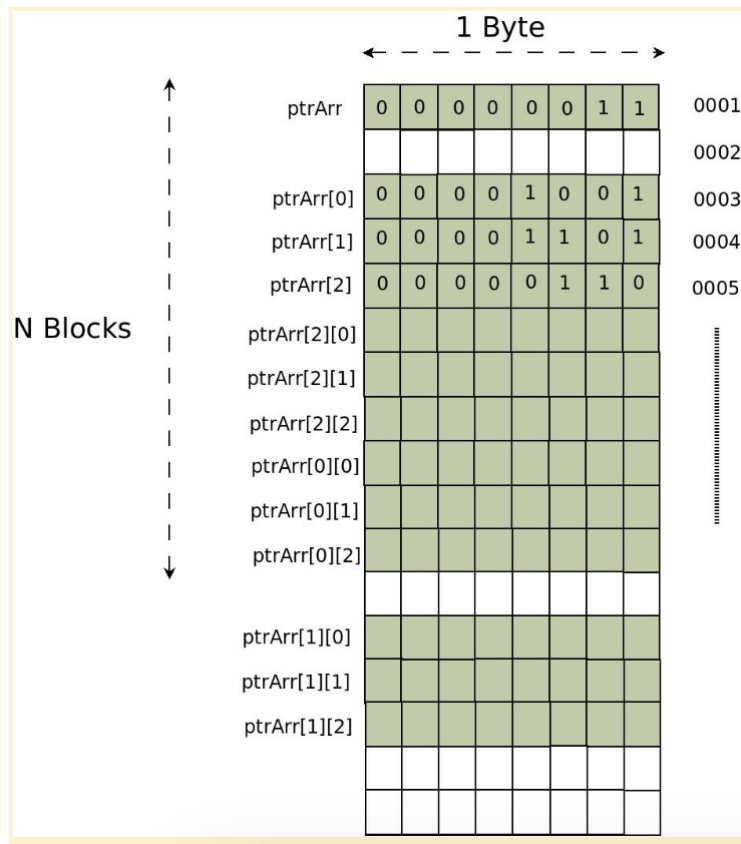
Summary: Memory for three pointers (ptrArr[0], ptrArr[1], and ptrArr[2]) and one double pointer (ptrArr) will be created.

(b) After executing Line 4 (after for loop completion), explain how the blocks of memory will be occupied by ptrArr[0], ptrArr[1], and ptrArr[2]. You need to clearly label and show the places in the memory block diagram in part (a) which will be affected by Line 3.

After executing line 4, memory will be reserved for the individual elements (columns) of a given row one after the other. First, the blocks for the first row (ptrArr[0][0], ptrArr[0][1], and ptrArr[0][2]) will be reserved contiguously, followed by second row (ptrArr[1][0], ptrArr[1][1], and ptrArr[1][2]) which is again followed by the third row. It should be noted that while reserving spaces for multiple rows, the blocks need not be contiguous.

Summary: Memory will be reserved for the individual elements of the 2D array (i.e., ptrArr[i][j] $\forall i, \forall j$)

(c) Clearly label and show the locations in the memory block diagram given in part (a) for the following pointer representations: ptrArr[1][1], ptrArr[2][0], *(ptrArr+1), *(*ptrArr+0)+2



There are many ways to allocate memory for the given code snippet. As at the time of allocation/reserving memory, depending on the availability of the individual memory blocks, the space

will be reserved. However, the concept explained above should be understood and accordingly the labelling should be done.

- (b) (5 points) Given two sorted arrays, the task is to merge them in a sorted manner.

Input: `arr1[ ] = { 1, 3, 4, 5}`, `arr2[ ] = {2, 4, 6, 8}`

Output: `arr3[ ] = {1, 2, 3, 4, 4, 5, 6, 8}`

Input: `arr1[ ] = { 5, 8, 9}`, `arr2[ ] = {4, 7, 8}`

Output: `arr3[ ] = {4, 5, 7, 8, 8, 9}`

The `mergeArrays` function has some bugs; your task is to rewrite the `mergeArrays` function after fixing the bugs

```
1 #include <stdio.h>
2 // Merge ar1[0..n1-1] and ar2[0..n2-1] into ar3
3 void mergeArrays(int* ar1, int n1, int* ar2, int n2, int* ar3
4 ) {
5     int i = 0, j = 0, k = 0;
6     while (i < n1 || j < n2)
7     {
8         if (ar1[i] < ar2[j]) ar3[k++] = ar1[i++];
9         else ar3[k++] = ar2[j++];
10    }
11    while (i < n1) ar3[k++] = ar1[i++];
12    while (j < n2) ar3[k++] = ar2[j++];
13 }
14 int main() {
15     int ar1[] = {1, 3, 5, 7};
16     int ar2[] = {2, 4, 6, 8};
17     int n1 = sizeof(ar1) / sizeof(ar1[0]);
18     int n2 = sizeof(ar2) / sizeof(ar2[0]);
19     int ar3[n1 + n2];
20     mergeArrays(ar1, n1, ar2, n2, ar3);
21     for (int i = 0; i < n1 + n2; i++) printf("%d ", ar3[i]);
22     return 0;
23 }
```

Listing 5: Fix the bugs

```
1 #include <stdio.h>
2 // Merge ar1[0..n1-1] and ar2[0..n2-1] into ar3
3 void mergeArrays(int* ar1, int n1, int* ar2, int n2, int* ar3
4 ) {
5
6     {
7
8     }
9 }
```



```

10
11
12 }

```

Listing 6: Your solution

```

1 #include <stdio.h> // There are corrections in 5 lines so 1
   mark for each
2 // Merge ar1[0..n1-1] and ar2[0..n2-1] into ar3
3 void mergeArrays(int* ar1, int n1, int* ar2, int n2, int* ar3
   ) {
4     int i = 0, j = 0, k = 0;
5     while (i < n1 && j < n2) // || was the bug.
6     {
7         if (ar1[i] < ar2[j]) ar3[k++] = ar1[i++]; // It was
           picked from ar2.
8         else ar3[k++] = ar2[j++]; // It was picked from arr1.
9     } // Pick smaller of the two current elements and move
       ahead in the array of the picked element
10    while (i < n1) ar3[k++] = ar1[i++]; // if there are
       remaining elements of the first array, move them. i should
       not be > n1, and i index is for ar1, not j.
11    while (j < n2) ar3[k++] = ar2[j++]; // Else if there are
       remaining elements of the second array, move them. j
       index is for ar2, not i.
12 }

```

Listing 7: Solution Key

- (c) (5 points) In this game of Book Cricket, Tom and Jerry take turns to "play" using sequences of page numbers, where each page's last digit determines their score for that page. This book has pages only on the right side. Its left-side pages are blank. That means after randomly opening a book; a player can see all odd/even numbers. Tom plays first, followed by Jerry. Rules are:

- (1) End of Turn: If the last digit is 0, that player's turn ends.
- (2) Scoring: If the last digit is between 1 and 6, the player scores that many runs. If the last digit is greater than 6, the player scores 0 but continues their turn.

After both players complete their turns, they compare their total runs, and the player with the higher score wins. If both have the same score, the game is a draw.

The program receives two lines of input: The first line lists page numbers for Tom's turn. The second line lists page numbers for Jerry's turn. Each sequence contains a page number ending in 0 to signal the end of the player's turn.

The task is to debug the given program. Only necessary corrections are allowed, focusing on fixing compilation, runtime, and logical errors, without rewriting it from scratch.

Input: 23 45 67 80
34 12 76 90

Output: Jerry won

Explanation:

Tom's turn:

- (1) Page 23 → Last digit 3: Tom scores 3 runs.

- (2) Page 45 → Last digit 5: Tom scores 5 runs.
- (3) Page 67 → Last digit 7: Tom scores 0 runs but continues.
- (4) Page 80 → Last digit 0: Tom's turn ends.

Jerry's turn:

- (1) Page 34 → Last digit 4: Jerry scores 4 runs.
- (2) Page 12 → Last digit 2: Jerry scores 2 runs.
- (3) Page 76 → Last digit 6: Jerry scores 6 runs.
- (4) Page 90 → Last digit 0: Jerry's turn ends.

Result:

Tom's total score: $3 + 5 + 0 = 8$

Jerry's total score: $4 + 2 + 6 = 12$

Winner: Jerry wins.

Line No	Corrected Code
1	#include <stdio.h>
2	int main() {
3	int num;
4	int tScore = 0;
5	int jScore = 1;
6	do{
7	scanf("%d", &num);
8	if(num == 0) break;
9	if(num <= 6)
10	jScore += num;
11	}while(1)
12	do{
13	scanf("%d", &num);
14	if(num <= 0) break;
15	if(num <= 6)
16	tScore += num;
17	}while(1)
18	if(tScore > jScore)
19	printf("Tom won");
20	if(tScore < jScore)
21	printf("Jerry won");
22	if(tScore = jScore)
23	printf(Draw);
24	return 0;
25	}
DO NOT SUGGEST CORRECTIONS TO MORE THAN 10 LINES	

```

1 #include <stdio.h> // 0.5 marks for each correction
2 int main(){
3     int num;
4     int tScore = 0;
5     int jScore = 0; // Initial score should be 0
6     do{
7         scanf("%d", &num);
8         if(num%10 == 0) break; // Need to take mode here
9         if(num%10 <= 6) // Need to take mode here
10            tScore += num; // This is toms score

```

```

11     }while(1); // semicolon missing
12     do{
13         scanf("%d", &num);
14         if(num%10 == 0) break; // Need to take mode here
15         if(num%10 <= 6) // Syntax error and mode
16             jScore += num; // this is jerry's score
17     }while(1); // semicolon missing
18     if(tScore > jScore)
19         printf("Tom won");
20     if(tScore < jScore)
21         printf("Jerry won");
22     if(tScore == jScore) // This should be comparison
23         printf("Draw"); // Double quotes are missing
24     return 0;
25 }

```

Listing 8: Solution Key

- (d) (4 points) Write down the output for the given input in the space provided. The output has four lines. Assume that `stdio.h` and `stdlib.h` have been included.

```

int main(){
int **a = (int**)malloc(8);
*a = (int*)malloc(sizeof(int));
**a = 1;
int **b = (int**)malloc(8);
*b = (int*)malloc(sizeof(int));
**b = 2;
int **c = (int**)malloc(8);
*c = (int*)malloc(sizeof(int));
**c = 3;
a = c;
printf("%d %d %d\n",**a,**b,**c);
*c = *b;
printf("%d %d %d\n",**a,**b,**c);
*b = *c;
printf("%d %d %d\n",**a,**b,**c);
printf("%d", (**a)*(**b)*(**c));
return 0;
}

```

Output:

```

3 2 3
2 2 2
2 2 2
8

```

NOTE: 1 mark for each line

4. Long answers.

- (a) (18 points) The definition of the Node structure is given as `typedef struct Node{ int value; struct Node *next; }NODE;`. Your task is to write the functions described below to manipulate a singly linked list. Assume that `stdio.h` and `stdlib.h` are included. 3 marks for each function.
- (1) `NODE* make_node(int val);` Allocates and returns new node pointer and sets `val` to its `value` field, `next` field is `NULL`.
 - (2) `NODE *insert_front(int val, NODE* head)` : inserts new node at the `head` of the list and returns pointer to the head of the new list.
 - (3) `void recursive_print(NODE* head);` This function prints the given list in the reverse order.
 - (4) `NODE* search(NODE* head, int key)` : search for the `key` in a list and return a pointer to the node found or else return `NULL`.

(6) You have been given the correctly written main function, which uses all the above-mentioned functions. Your task is to write the output of this main function if a user inputs 5 4 3 2 1 -1

```

NODE *insert_front(int val, NODE*
head) {

```

```
NODE* delete_node(NODE* pnode, NODE*
ppnode) {
```

```
int main() {
    NODE* head = NULL;
    int n; scanf ("%d", &n);
    while (n != -1) {
        head = insert_front (n, head);
        scanf ("%d", &n);
    }
    recursive_print(head);
    NODE* pppnode=head->next;
    NODE* pNode=head->next->next;
    head=delete_node(pNode, pppnode);
    recursive_print(head);
    if(search(head,1))
        printf("Found");
    else printf("Not found");
    return 0; }
```

Output:

5 4 3 2 1

5 4 2

Not found

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 typedef struct node{
4     int value;
5     struct node *next;
6 } NODE;
7 // 3 marks for each block.
8
9 NODE* make_node(int val)
10 {
11     NODE* temp = (NODE*) malloc(1*sizeof(NODE));
12     temp->value = val;
13     temp->next = NULL;
14     return temp;
15 }
16
17 NODE *insert_front(int val, NODE *head)
18 {
19     NODE *newnode= make_node(val);
20     newnode->next = head;
21     head = newnode;
22     return head;
23 }
24
25 NODE* search(NODE* head, int key)
26 {
27     NODE* curr = head;
28     while(curr != NULL && curr->value != key) curr = curr->
next;
29     return curr;
30 }
31
32 NODE* delete_node(NODE* pnode, NODE* ppnode) {
33     NODE* t;
34     if (ppnode) ppnode->next = pnode->next;
35     t = ppnode ? ppnode : pnode->next;
36     free(pnode);
37     return t;
38 }
39
40 void recursive_print(NODE* head)
```

```

41 {
42     NODE* temp = head;
43     if(temp==NULL) return;
44     recursive_print(temp->next);
45     printf("%d ",temp->value);
46 }
47
48 int main(){
49     NODE* head = NULL;
50     int n;scanf("%d",&n);
51     while(n!=-1){
52         head = insert_front (n, head);
53         scanf("%d",&n);}
54     recursive_print(head);
55     printf("\n");
56     NODE* pnode = head->next;
57     NODE* ppnode = head->next->next;
58     head = delete_node(pnode, ppnode);
59     recursive_print(head);
60     if(search(head,1)) printf("\nFound");
61     else printf("\nNot found");
62     return 0;
63 }

```

Listing 9: Solution for Linked list module

- (b) (7 points) **ZIG-ZAG Traversal with Recursion** Assume that you have a matrix `arr` with (N,M) dimensions. You need to write a recursive function named ZIG-ZAG that traverses this matrix in the zig-zag order. You will start at (0,0). If the row index is even, then you can move to the right; if it is odd, you can move to the left. In this fashion, you need to traverse till you visit all the elements in the matrix. For example, the output of `arr = {{1,2},{3,4}}` will be 1 2 4 3 and the output of `arr = {{1,2,3},{4,5,6},{7,8,9}}` will be 1 2 3 6 5 4 7 8 9.
NOTE: No marks will be given if you do it without recursion. You may lose marks if don't use proper indentation and give comments to tell your logic.

```

1 void ZIG-ZAG(int arr[][M], int i, int j) // 1 mark for
    correct function definition and there are 6 cases so 1
    mark for each
2 {
3     //Base case
4     if(N%2 == 1 && i== N-1 && j == M-1){printf("%d ",arr[i][j]
    ); return;}
5     if(N%2 == 0 && i== N-1 && j == 0){printf("%d ",arr[i][j])
    ; return;}
6
7     // Recursive case
8     if (j<M-1 && i%2==0) {printf("%d ",arr[i][j]);traverse(
    arr , i , j+1);}

```

```

9     else if (j>0 && i%2==1) {printf("%d ",arr[i][j]);traverse
    (arr,i,j-1);}
10    else if (i<N-1 && i%2==0) {printf("%d ",arr[i][j]);
    traverse(arr,i+1,M-1);}
11    else if (i<N-1 && i%2==1) {printf("%d ",arr[i][j]);
    traverse(arr,i+1,0);}
12 }

```

Listing 10: Zig-zag solution.

- (c) (10 points) You have an "input.txt" file containing "Welcome CS101" (without double quotes) and an empty "output.txt" file. Complete the following code based on the provided comments. Implement a function declared as `void copy_file(FILE* f, FILE* t);` that copies the contents of `f` to `t`. Finally, write the terminal output in the given box. Do not use the standard `printf` to display the content on the terminal; use only file handling functions.

```

1 #include <stdio.h>
2 int main()
3 {
4     char buffer[10];
5     // open "input.txt"/"output.txt" in "r"/"w" modes resp.
6
7     FILE* f = -----
8
9     FILE* t = -----
10
11    fscanf(f,"%s\n",buffer);
12    // Display the content of buffer on the terminal
13
14    -----
15
16    // Write "Bye" (without double quotes) to the file t
17
18    -----
19
20    copy_file(f,t);
21    // Move the file pointer of t back by 5 positions
22
23    -----
24
25    // Write "Mr.C :(" (without double quotes) to the file t
26
27    -----
28
29    // Close both the files
30
31    -----
32

```

```

33  -----
34
35
36  // Open "output.txt" file again in "r" mode
37
38  FILE* temp = -----
39
40  fgets(buffer, sizeof(buffer), temp);
41
42  // Display the content of the buffer on the terminal
43
44  -----
45
46  return 0;
47
48 }

```

Listing 11: File handling

```

void copy_file(FILE *f, FILE *t) {
    while(fscanf(f, "%c", &ch) == 1) fprintf(t, "%c", ch);
}

```

Output:

Welcome
Bye Mr.C:
NOTE: 1.5 marks for each correct line.

```

1  #include <stdio.h>
2  void copy_file(FILE* f, FILE* t) // 2 marks
3  {
4      char ch;
5      while(fscanf(f, "%c", &ch) == 1) fprintf(t, "%c", ch);
6  }
7
8  int main() // 0.5 marks for each blank
9  {
10     // Write a copy_file function
11     char buffer[10];
12     FILE* f = fopen("./input.txt", "r");
13     FILE* t = fopen("./output.txt", "w");
14     fscanf(f, "%s", buffer);
15     // Display the content of buffer on terminal using
    fprintf
16     fprintf(stdout, "%s\n", buffer);
17     // Write "Bye" (without double quotes) to the file t
18     fputs("Bye", t);

```



```

19     copy_file(f,t);
20     // Move the file pointer of t back by 5 positions
21     fseek(t,-5,SEEK_CUR);
22     // Write "Mr.C :(" (without double quotes) to the file t
23     fputs("Mr.C:(",t);
24     // Close both the files
25     fclose(f);
26     fclose(t);
27     // Open output file in "r" mode
28     FILE* temp = fopen("./output.txt","r");
29     fgets(buffer,sizeof(buffer),temp);
30     fprintf(stdout,"%s",buffer);
31     return 0;
32 }

```

Listing 12: Solution of File handling